

He found that this light fixture burned out and was trying to tell the hub that it needed attention. However, when doing this, it was continuously sending requests to the hub, which overloaded the network and eventually caused it to freeze. In other words, it was causing a denial of service (DoS) to the rest of the smart home devices by standing in their way of communication with the hub. Rojas changed the bulb, and the problem vanished promptly.

This issue, however, highlights a few potential problems in smart homes as well as so-called autonomous systems. When things go wrong and are out of hands, how can an end-user take over control and put things back in their right place, let alone control or even investigate the matter to figure out what is wrong in the first place? Rojas did not design for a bypass mechanism where he could commandeer the whole system. Maybe because it was an integration of several heterogeneous systems, or perhaps he did not think of it.

Australian telco flushes millions of dollars

In early 2018, an Australian telecommunications company bit the bullet and rolled out an automation bot (auto-bot) for its incident handling process. Although the telco expected to reap benefits from the beginning of its implementation and save on more than 25 per cent of the operational costs, the plan backfired.

The auto-bot was designed to intercept all network incidents and then follow a series of checks based on the problem type selected by users. It was programmed to take one of the three actions based on tests it would perform. It would remotely resolve the incident by fixing the issue programmatically, or it would assume that a technician's visit is required to customer premises. And accordingly, it would send someone directly, or if none of that were apparent, it would present the case to the human operator for further investigation and decision.

This approach was sound and seemed quite logical in the first place. However, after commencement of the auto-bot operation, within a few

weeks, the company realised the auto-bot was sending an awful lot of technicians in the field than before. Sending out technicians for field visits was a costly affair and was always the last choice for fixing an issue. However, the auto-bot maximised on that.

After investigation, the team figured out that there were a few incident scenarios that a human operator could understand (and invariably join the dots) but were not clear enough to be programmed for the auto-bot. In all such cases, a human operator would have taken a different decision than the auto-bot.

The primary issue was, despite finding out the flaw in logic, the automation team was unable to turn off the auto-bot (much like what Microsoft did with Tay). They had implemented the auto-bot in all or nothing manner, and it was sitting right in the middle of the user and operator interface. It essentially meant, either all incidents would go through the auto-bot and get wrongly handled often, or none would go through the auto-bot and would get manually handled. Now, the only issue was that the telco was not ready to handle such a workload—they had already released the staff for saving costs.

Eventually, the telco set up another project to fix the auto-bot while it was in operation and wasted several million dollars in the process. They spent money on two things—for continuing service with the artificially-stupid auto-bot and for running a massive fix-up project that lasted more than a year.

Eventually, the endowment effect kicked in. The company had no plans to go back and fix the problem from its roots but instead kept pushing through and wasting an enormous amount of money. The crucial question is: who eventually paid for this?

In my view, this implementation went wrong on several levels, right from system design to its implementation and fixing of the problems. **But the first and foremost question that emanates is: Why there was no plan B, a kill switch of some sort to stop this auto-bot?**

The auto-bot development and rollout were not thoroughly tested for all potential scenarios and, thus,

lacked testing rigour, which could have identified problems early on. While the time required to fix the situation was too long, detecting the failure of the auto-bot took considerably longer.

Did you notice a pattern here?

Whether it is chatbot Tay by Microsoft, Uber's autonomous car or the auto-bot of Australian telco, all of these have something in common that failed. And it is not the technology!

In all these scenarios, either the creator of AI or businesses that deployed the AI was not careful enough. They did not follow the fundamental tenet of handling something as powerful as automation or primitive AI responsibly. We often say, "With great power comes great responsibility." And yet, in all these cases, responsible design or deployment did not happen or did not occur in full spirit.

Responsible behaviour is necessary for the deployment and use of AI as well as all other stages from conception to design, testing to implementation, and ongoing management and governance.

Almost all the cases discussed here had a certain level of weaknesses in the solution conception stage, and this directly seeped into their development.

In cases of a chatbot, telco's auto-bot and autonomous car, emphasis on solution quality was not enough. There might have been a few testing routines—just enough to meet the requirements of IT development frameworks—but not enough to meet the AI development framework, which does not exist!

In the case of Amazon, I would question the skillset of decision-makers. Although it appeared to be an inadvertent decision to roll out same-day deliveries to only a few localities, thoughtful consideration could have raised these questions in reflection. The same goes for the telco case or lightbulb situation where creators lacked thoughtfulness in the design of the solution.

Nine things to learn from this

While there are several use-cases of AI to learn from, I specifically chose these